

C++ Programming Textbook Notes

1. Keywords in C++

Conceptual Explanation

Keywords are pre-reserved words recognized directly by the C++ compiler. Each keyword has a special, fixed meaning and serves as a fundamental building block for constructing program logic, control flow, and data types.

When C++ was developed as an extension of C, it inherited C's core keywords (such as `int`, `if`, `while`, and `return`) and introduced new keywords (such as `class`, `public`, `template`, `namespace`, `constexpr`, and `auto`) to support Object-Oriented Programming (OOP), templates, modern memory management, and stronger type checking.

In memory, keywords themselves do not allocate storage. Instead, they instruct the compiler *how* to set up structural components, allocate variable memory, or execute instructions.

Key C++ Keywords (Comparison with C)

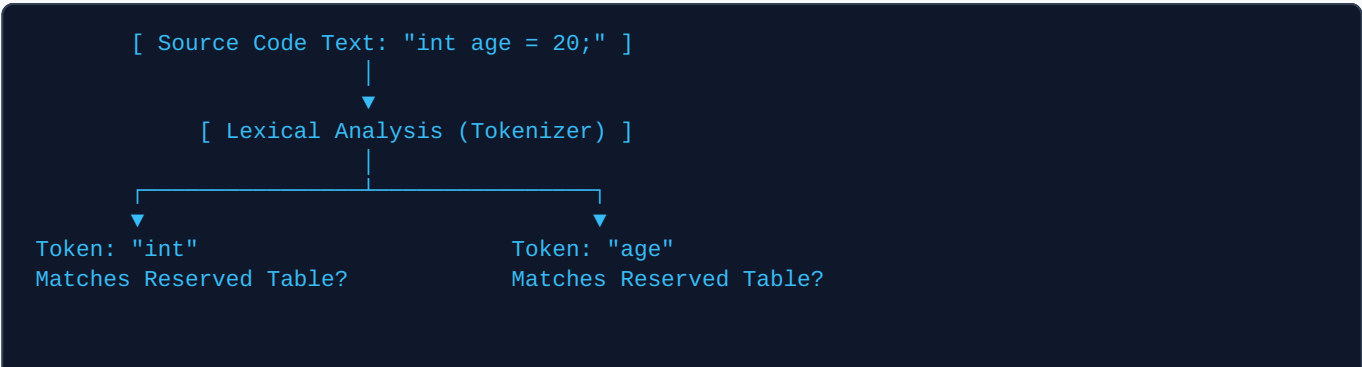
While standard ANSI C features 32 keywords, modern C++ includes over 90 keywords. Below is a subset comparing core C keywords with key modern C++ additions:

Category	C Keywords (Inherited)	C++ Specific Additions
Data Types	<code>int</code> , <code>float</code> , <code>char</code> , <code>double</code> , <code>void</code>	<code>bool</code> , <code>wchar_t</code> , <code>char16_t</code> , <code>auto</code>
Control Flow	<code>if</code> , <code>else</code> , <code>for</code> , <code>while</code> , <code>switch</code> , <code>case</code> , <code>break</code>	<code>try</code> , <code>catch</code> , <code>throw</code>
Object-Oriented	(None)	<code>class</code> , <code>public</code> , <code>private</code> , <code>protected</code> , <code>virtual</code> , <code>this</code>
Memory / Utility	<code>static</code> , <code>const</code>	<code>new</code> , <code>delete</code> , <code>constexpr</code> , <code>namespace</code> , <code>using</code> , <code>nullptr</code>

Syntax & Rules

- Lowercase Only:** C++ keywords are written strictly in lowercase (e.g., `int` is valid; `Int` or `INT` will fail).
- Reserved Status:** Keywords **cannot** be used as variable names, function names, or class identifiers.

Text-Based Flowchart: Compiler Keyword Parsing



└ YES: Keyword!
└ Action: Set type = Integer

└ NO: Valid Identifier!
└ Action: Register in Symbol Table

Code Example

```
// Keywords demonstration in C++
#include <iostream>

int main() {
    // 'int', 'const', 'bool', and 'return' are C++ keywords
    const int maxUsers = 100;
    bool isActive = true;

    if (isActive) { // 'if' is a control flow keyword
        std::cout << "Max allowed users: " << maxUsers << std::endl;
    }

    return 0; // 'return' exits the function
}
```

⚠ **Common Gotcha:** Capitalizing keywords. Writing `If (x > 0)` or `Int count = 5;` will throw compilation errors.

💡 **Best Practice:** Modern IDEs highlight keywords in distinct colors. If a word does not highlight, double-check its spelling and capitalization.

2. Identifiers in C++

Conceptual Explanation

An **Identifier** is a user-defined name given to program elements such as variables, functions, arrays, structures, classes, and namespaces.

Just as real-world names (e.g., *Delhi*, *Gaurav*) identify specific places or people, identifiers in C++ give human-readable names to underlying memory addresses, code blocks, or custom data structures.

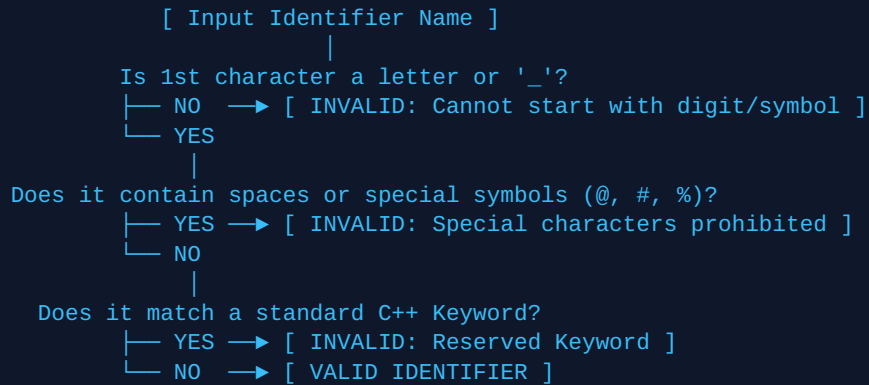
Real-World vs. C++ Identifiers

- **Real World:** Student Name (Gaurav), City (Delhi)
- **C++ Program:** `studentAge` (Variable), `calculateSalary()` (Function), `Student` (Class)

Syntax & Rules (The 5 Golden Rules)

1. **Allowed Characters:** Must start with an uppercase/lowercase letter (a-z, A-Z) or an underscore (`_`). It may be followed by letters, digits (0-9), or underscores.
2. **No Leading Digits:** An identifier **cannot** begin with a number (e.g., `1age` is invalid).
3. **No Whitespace:** Spaces are strictly prohibited (e.g., `student age` is invalid; use `student_age` or `studentAge`).
4. **No Special Symbols:** Symbols like `@`, `#`, `%`, `$`, and `&` are forbidden.
5. **Case Sensitivity:** C++ is strictly case-sensitive. `age`, `Age`, and `AGE` represent three distinct identifiers in memory.

Text-Based Flowchart: Identifier Validation



Code Example

```
#include <iostream>

// Class Identifier
class BankAccount {
public:
    double balance; // Variable Identifier
};

// Function Identifier
void displayMessage() {
    std::cout << "Welcome to C++ Programming!" << std::endl;
}

int main() {
    int user_age = 20;    // Valid identifier (Snake Case)
    int userAge = 20;     // Valid identifier (Camel Case)
    int USAGE = 20;       // Valid, but distinct from userAge due to case sensitivity

    displayMessage();
    return 0;
}
```

 **Common Gotcha:** Accidental double underscores (`__`) or starting an identifier with an underscore followed by a capital letter (e.g., `_MyVariable`). C++ standard libraries reserve identifiers starting with underscores for internal implementation. Avoid using leading underscores in user code.

 **Best Practice:** Use **camelCase** for variables/functions (`studentAge`, `calculateTotal`) and **PascalCase** for classes/structs (`BankAccount`, `CustomerRecord`).

3. Variables in C++

Conceptual Explanation

A **Variable** is a named storage location in physical RAM assigned to hold data values that can be read or modified during program execution.

When you declare a variable, the compiler allocates a specific block of bytes based on the variable's **Data Type**. The variable's identifier acts as a convenient, symbolic label for that underlying raw memory address.

Text-Based Memory Diagram

Variable Label: 'age'
Data Type : int (4 Bytes)
Value Stored : 20

RAM Layout:

Address	Memory Contents	
0x7ffd9000	[00000000]	} 4 Bytes reserved for int 'age' storing 20
0x7ffd9001	[00000000]	
0x7ffd9002	[00000000]	
0x7ffd9003	[00010100]	

Declaration vs. Initialization vs. Assignment

- Declaration:** Tells the compiler the name and data type of the variable without necessarily putting a meaningful value inside it.
`int age; // Declared, contains garbage data until written`
- Initialization:** Assigns an initial value at the time of creation.
`int age = 20; // Copy initialization`
- Modern C++ Uniform Initialization (Direct List Initialization):**
`int age{20}; // Prevents implicit narrowing conversions (C++11 standard)`
- Assignment:** Overwriting an existing variable's value later in the execution flow.
`age = 21; // Modifies the value in memory from 20 to 21`

Categories of Variables by Scope & Lifetime

Type	Scope (Visibility)	Lifetime
1. Local Variable	Inside defining block { }	Created at block entry, destroyed at block exit
2. Global Variable	Entire file / cross-file	Created at app start, destroyed at app exit
3. Static Local Variable	Inside defining block { }	Created once on first call, persists until app exit

Code Example

```
#include <iostream>

// 1. Global Variable: Accessible anywhere in the program
int globalCounter = 100;

void processCalls() {
    // 2. Static Variable: Retains its value across multiple function calls
    static int callCount = 0;

    // 3. Local Variable: Re-created on each call, destroyed when function ends
    int tempVal = 10;
```

```

    callCount++;
    tempVal++;

    std::cout << "Call #" << callCount << " | Temp: " << tempVal << std::endl;
}

int main() {
    // Local variable using C++11 list initialization
    int age{20};

    std::cout << "Initial age: " << age << std::endl; // 20
    age = 21; // Value reassignment
    std::cout << "Updated age: " << age << std::endl; // 21

    std::cout << "Global Counter: " << globalCounter << std::endl;

    processCalls(); // Call #1 | Temp: 11
    processCalls(); // Call #2 | Temp: 11 (callCount retained, tempVal reset)

    return 0;
}

```

 **Uninitialized Local Variables:** Declaring a local variable without initialization (e.g., `int score;`) leaves raw bit patterns ("garbage values") in memory. Reading uninitialized local variables leads to undefined behavior.

 **Narrowing Conversions:** Traditional syntax `int x = 3.99;` silently truncates the decimal portion to 3. Modern brace initialization (`int x{3.99};`) triggers a compilation warning/error to protect against unintended data loss.

 **Best Practice:** Always initialize variables immediately upon declaration, keep variable scope as narrow as possible, and prefer modern C++ `{}` initialization.